

HealthTrack Patient Monitoring System

Final Technical Report

MSCS-699 — Capstone Project
University of the Cumberlands

Author: Arun Bhaskar Gadde
Final Project Submission

Table of Contents

1. Executive Summary
 2. System Architecture and Design
 3. Database Design
 4. Risk Assessment Methodology
 5. Alert Management Design
 6. Testing Summary
 7. Known Limitations
 8. Deployment Overview
 9. Design Decisions and Challenges Encountered
 10. Conclusion and Recommendations
- References

1. Executive Summary

HealthTrack is a patient monitoring system built across a series of capstone deliverables, culminating in this final, integrated submission. It ingests patient vital signs, runs a combined rule-based and machine-learning risk assessment on each reading, raises deduplicated and escalating alerts for at-risk patients, and displays open alerts on a real-time dashboard.

This report documents the final system as actually built and tested: architecture, database design, the risk assessment methodology, the alert management design, and the results of unit, integration, User Acceptance, and manual system testing. Numbers throughout this report — test pass counts, coverage percentages, model performance — are taken from real, freshly captured runs of the system rather than estimated or written to look ideal. Where something did not work or was not verified, that is stated plainly rather than glossed over; see Section 7, Known Limitations.

At the time of this submission: 26 of 26 automated unit and integration tests pass, 8 of 8 User Acceptance Test scenarios pass, code coverage on the application package is 97%, and the local/single-server deployment path has been verified end-to-end from a clean environment. The Docker Compose deployment path is written and syntax-validated but not verified, since Docker was unavailable in the build environment — this is disclosed rather than hidden.

2. System Architecture and Design

HealthTrack follows a layered architecture with four independently deployable components:

Component	Technology	Responsibility
Backend API	FastAPI + SQLAlchemy	Patient, vitals, risk assessment, alert, and staff endpoints; orchestrates the risk engine and alert manager on every vitals reading
Database	PostgreSQL (prod) / SQLite (dev)	Persists patients, vitals, risk assessments, alerts, staff, devices, and notes
Risk engine	NEWS2 rules + scikit-learn Random Forest	Scores each vitals reading and classifies risk level
Dashboard	Dash (Plotly) + Flask	Polls the API and displays the live open-alert queue to clinical staff

2.1 Data flow

A device (or simulated feed) posts a vitals reading to POST /vitals. The API persists the reading, immediately runs it through the risk engine (Section 4), persists the resulting risk assessment, and — if the risk level warrants it — creates an alert through the deduplication logic in the alert manager (Section 5). The API then broadcasts the outcome over a WebSocket channel (/ws/alerts) to any connected dashboard clients, and the Dash dashboard separately polls GET /alerts every 5 seconds as a simpler, more resilient path that degrades gracefully if the WebSocket connection drops.

2.2 Why this separation

The dashboard is a separate process from the API by design, communicating only over HTTP/WebSocket rather than sharing in-process state. This was verified directly: the dashboard was run against a live API in a separate process during testing, including a scenario where the API was unreachable, and the dashboard displayed a clear connection-error message rather than crashing. This separation also allows

the dashboard to be restarted, redeployed, or scaled independently of the API — relevant to the maintenance guidance in Section 8.

3. Database Design

The full HealthTrack entity-relationship design — 20 entities — was produced in an earlier database design deliverable of this capstone. This final integration implements and exercises the working subset of that design needed for the end-to-end patient monitoring flow: 7 tables, shown below.

Table	Purpose	Key relationships
patients	Core patient demographic record	1—many vitals, assessments, alerts
staff	Clinical/admin users who acknowledge alerts	1—many acknowledged alerts
devices	Monitoring hardware associated with a patient	many—1 patient
vital_signs	Individual vitals readings	many—1 patient, many—1 device
risk_assessments	NEWS2 + model output per reading	many—1 patient, many—1 vital_signs
alerts	Deduplicated, escalatable alerts	many—1 patient, many—1 risk_assessment, many—1 staff
patient_notes	Free-text clinical notes	many—1 patient, many—1 staff

Indexes were added on (patient_id, timestamp) for both vital_signs and risk_assessments, since "recent history for this patient" is the dominant query pattern for both the dashboard and the physician-review workflow (see UAT-6 in Section 6). Schema changes are managed through Alembic migrations rather than ad hoc DDL; two migrations exist at the time of this report — the initial schema, and the addition of patient_notes — both verified with a full upgrade/downgrade/re-upgrade cycle (Section 8 has the walkthrough).

4. Risk Assessment Methodology

Every vitals reading is scored two ways, and the two scores are combined into a final risk level:

4.1 NEWS2 — rule-based scoring

The National Early Warning Score 2 (Royal College of Physicians, 2017) is a clinically established, transparent scoring system: each vital sign (respiratory rate, SpO2, systolic blood pressure, heart rate, temperature, consciousness level) is scored against clinical bands, and the points are summed into an aggregate score. HealthTrack implements a simplified version of these bands in app/risk_engine.py. Its main advantage is that a clinician can look at a patient's NEWS2 score and immediately understand why it is what it is — there is no black box.

4.2 Random Forest — learned probability

A Random Forest classifier, trained on a synthetic 2,000-patient vitals dataset (scripts/train_risk_model.py), predicts the probability that a given set of vitals belongs to a "High risk" patient. On its own held-out test set, it reaches 88% overall accuracy but only 0.54 recall on the High-risk

class specifically — meaning it misses a meaningful share of true high-risk cases if used alone. This is reported honestly in Section 6.4 rather than leading with the flattering overall-accuracy number.

4.3 Combining the two

The final risk level is NOT the model's output alone. A NEWS2 score of 7 or higher always produces a Critical classification regardless of the model, since that threshold is clinically established. For borderline NEWS2 scores (3–6), the Random Forest's probability is used to decide whether to round up or down between Moderate and High. This design was a deliberate choice to compensate for the model's imperfect recall with a transparent, clinically-grounded floor — the system does not rely on the model to catch severity that the rules already catch on their own.

```
if news2_score >= 7:
    risk_level = CRITICAL
elif news2_score >= 5:
    risk_level = HIGH if model_probability >= 0.6 else MODERATE
elif news2_score >= 3:
    risk_level = MODERATE if model_probability < 0.5 else HIGH
else:
    risk_level = LOW
```

5. Alert Management Design

5.1 Deduplication

Without deduplication, a patient sitting at Critical risk would generate a new alert on every single vitals reading — potentially every few seconds from a continuous monitor — which would flood staff and, in practice, train them to ignore alerts entirely. HealthTrack deduplicates on a (patient, risk level) key: if an open or escalated alert with the same key was created within the last 15 minutes, a new one is not created. This was directly verified in UAT-4, where three consecutive critical readings for the same patient produced exactly one open alert, not three.

5.2 Escalation

An alert that sits Open and unacknowledged for more than 10 minutes is escalated. Escalation is implemented as an explicit endpoint (POST /alerts/escalate-stale) rather than a background timer inside the API process, so it must be triggered externally on a schedule (cron, or an application scheduler) — this is called out clearly as an operational requirement in the Maintenance Guide, since it is easy to assume incorrectly that escalation "just happens."

5.3 Acknowledge / resolve lifecycle

Alerts move through Open -> Acknowledged -> Resolved (or Open -> Escalated -> Acknowledged -> Resolved). Acknowledging records which staff member took ownership, which both clears the alert from the open queue and creates a minimal accountability trail — though, as noted in Section 7, this is not a substitute for a real audit log, which does not yet exist.

6. Testing Summary

Full detail, including raw output and coverage tables, is in the companion Test Results and Analysis document. This section summarizes the headline results, all captured from a single fresh run immediately before this report was finalized.

6.1 Unit and integration testing

Suite	Tests	Result
Risk engine (test_risk_engine.py)	9	9 passed
Alert manager (test_alerts.py)	7	7 passed
API / integration (test_api.py)	10	10 passed
Total	26	26 passed, 0 failed

Code coverage on the app package: 97% (408 statements, 14 missed). The missed lines are defensive fallbacks and exception paths not on the primary vitals -> risk -> alert flow — for example, the WebSocket disconnect-during-broadcast path, and the "no trained model present" fallback in the risk engine, which is not exercised because the model file is present in this environment as it should be in any real deployment.

6.2 User Acceptance Testing

Eight scripted clinical workflows were run against the API the way an actual client would call it (not through direct database or function access): patient registration, normal vitals producing no alert, critical vitals producing an alert, deduplication under repeated critical readings, nurse acknowledgment, physician history review, duplicate-MRN rejection, and stale-alert escalation. All 8 passed.

6.3 Manual system / integration testing

Beyond the automated suites, the system was run as real, separate processes (not test-client mocks): the API and dashboard were started independently, vitals were posted via curl to the live API, and the dashboard was confirmed to pull and display the resulting alert over real HTTP. The Alembic migration chain was run forward and backward twice against a real SQLite file. A completely clean deployment (fresh virtual environment, no pre-existing database or trained model) was run through setup.sh and run.sh end to end, surfacing and fixing one real environment issue along the way (Section 9.2).

6.4 Risk model performance — an honest read

The Random Forest reaches 88% overall accuracy, but that number is inflated by class imbalance (roughly 19% of the synthetic dataset is "High" risk). The metric that actually matters for a monitoring system — recall on the High-risk class — is 0.54, meaning the model alone would miss a substantial share of true high-risk patients. This is precisely why the risk engine does not use the model in isolation (Section 4.3). The model has been validated only against its own synthetic held-out test set, not against real clinical outcomes; see Known Limitations.

7. Known Limitations

Listed in priority order, and carried through in full in the Handover Documentation:

- No authentication or authorization exists yet. Every endpoint is currently open. This is the single most important gap to close before this system could handle real patient data.
- The risk model was trained on synthetic vitals data, not real clinical outcomes, and has not been independently validated.
- Alert escalation requires an external scheduler to call POST /alerts/escalate-stale; nothing triggers it automatically yet.
- Datetimes are naive UTC throughout by deliberate choice (see Section 9.1), which assumes a single-timezone deployment.
- There is no data retention or archival policy; vitals and risk-assessment tables grow unbounded.
- The Docker Compose deployment path is written and YAML-validated but was not actually run, since Docker was unavailable in the build environment.
- The dashboard has a single view (open alerts only); no per-patient trend view, ward filtering, or role-based views yet.
- There is no notification channel beyond the dashboard itself — no SMS, pager, or email integration for off-screen staff.

8. Deployment Overview

Full step-by-step instructions are in the companion Deployment Guide; this section summarizes what exists and what was verified.

8.1 Local / single-server path — verified

scripts/setup.sh creates a virtual environment, installs dependencies, runs migrations, and trains the risk model. scripts/run.sh starts the API and dashboard together. This full path was run in a clean directory with no pre-existing state, and both services came up and passed health checks while communicating with each other over real HTTP.

8.2 Docker Compose path — written, not verified

deployment/docker-compose.yml defines three services (Postgres, API, dashboard) with Dockerfiles for each. The compose file's YAML syntax and service structure were validated with a YAML parser, but Docker itself was not available in the build environment, so the containers were never actually built or run. This is flagged explicitly as a next step in Section 7 and the Handover Documentation rather than presented as done.

9. Design Decisions and Challenges Encountered

9.1 Naive vs. timezone-aware datetimes

During development, datetime.utcnow() calls throughout the codebase were briefly migrated to timezone-aware datetime.now(timezone.utc) to address a Python deprecation warning. This was reverted deliberately: the alert deduplication and escalation logic compares stored timestamps (naive, from SQLite) against freshly computed cutoffs, and mixing aware and naive datetimes in that comparison risks silently incorrect comparisons rather than a visible error. The deprecation warning was judged the lesser risk, and this is recorded here so a future maintainer does not "fix" the warning without understanding why it was left as-is.

9.2 A real environment bug caught during clean-deployment testing

Running scripts/setup.sh in a genuinely clean directory (no prior venv) surfaced an externally-managed-environment error — a real, common failure mode on modern Debian/Ubuntu systems that block global pip installs by default. The fix was to have setup.sh create and use a dedicated virtual environment rather than installing into the system Python. This was caught only because the deployment path was actually tested from a clean state rather than assumed to work.

9.3 A real modeling bug caught by an inconsistent metric

An early version of cross-validation for a related classification deliverable reported a cross-validated F1 score of roughly 0.91 that did not match the held-out test F1 of roughly 0.60 for the same class. The cause was that scikit-learn's default F1 scorer was treating the majority class as positive rather than the clinically relevant minority (High-risk) class. This was caught precisely because the two numbers were compared and found inconsistent, rather than each being accepted at face value — the same discipline was applied throughout this final deliverable's testing.

9.4 A model-merge bug caught by a nonsensical migration

While adding the patient_notes table during this final integration, an editing mistake merged two SQLAlchemy model class bodies together, silently deleting the Alert class declaration. This was caught because Alembic's autogenerate produced a migration proposing to add two unrelated columns to the alerts table instead of creating a new table — a nonsensical diff that prompted a direct inspection of the model file rather than blindly applying the generated migration. The full test suite (26 tests) was re-run after the fix and confirmed passing before the migration was regenerated correctly.

10. Conclusion and Recommendations

HealthTrack, as submitted, is a working, tested, end-to-end patient monitoring system: vitals ingestion, combined rule-based and ML risk assessment, deduplicated and escalating alerting, and a real-time dashboard, all verified through automated tests, User Acceptance scenarios, and manual live-process testing. It is not production-ready for real patient data as-is — principally because it has no authentication layer yet — and this report has tried throughout to be equally clear about what works and what doesn't.

The recommended next steps, in priority order, are: add authentication/authorization before any real data touches the system; stand up a scheduler for alert escalation; validate (or retrain) the risk model against real clinical outcomes rather than synthetic data; and actually run the Docker Compose path in a real Docker environment. These are detailed further, with rationale, in the Handover Documentation.

References

Royal College of Physicians. (2017). National Early Warning Score (NEWS) 2: Standardising the assessment of acute-illness severity in the NHS.

<https://www.rcp.ac.uk/improving-care/resources/national-early-warning-score-news-2/>

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.

Ramírez, S. (2018). FastAPI [Computer software]. <https://fastapi.tiangolo.com/>

SQLAlchemy authors. (n.d.). SQLAlchemy documentation. <https://docs.sqlalchemy.org/>

Plotly Technologies Inc. (n.d.). Dash documentation. <https://dash.plotly.com/>